

EUCR: An Evidential Continual Learner, and an Accounting of Which of Its Parts Do the Work

EUCR design note

Abstract

EUCR is the evidential analogue of EWC: a Dempster–Shafer (DS) prototype head on a 1D ResNet, with per-stage evidential probes whose uncertainty supplies a MAS-style consolidation importance. It works as a continual learner — on a four-task IQ radar sequence it reaches a final F1 of 0.596 against EWC’s 0.427, though it trails Learning-without-Forgetting’s 0.660 — but reaching that number required fixing two measurement faults and one numerical fault, and the accounting of *which* of its parts produce it is largely negative.

Three findings carry the note. First, EUCR was being scored on a different evaluation protocol from every model it was compared against: every `ResNet1D`-based learner reaches its backbone through a `forward` whose `bn_training` defaults to `True`, so the benchmark uses *batch* normalisation statistics, while EUCR — lacking that wrapper — used frozen running statistics. That asymmetry alone was worth 0.31 final F1. Second, the consolidation anchor’s loss form is outside its stability window at every λ tested: Ω is normalised to unit mean but has median 2.7×10^{-4} and maximum 1.7×10^4 , so a few tail coordinates make the anchor gradient outweigh the task gradient $60\times$ and the global gradient clip then attenuates the task signal to 1.6% of its size. A closed-form proximal anchor removes this, worth a further +0.07 final F1 and a $14\times$ reduction in seed variance. Third, and least comfortably for the method’s premise: comparing importance *frontiers* rather than points, a fixed random linear readout on the same stage features traces the same trade-off curve as the evidential one, reparameterised by a constant. The per-coordinate weighting contributes (+0.017 over uniform Ω); nothing about it needs to be learned, and nothing about it needs to be evidential.

We also record the closed-form Dempster fusion that makes the head affordable ($40\text{--}90\times$ on that layer, $6.6\times$ end-to-end, outputs matching the sequential loop to 2×10^{-6}), a construction-level pathology in which the ignorance mass is identically zero, a pre-registered gate in which neither the ignorance nor the conflict readout beats a softmax baseline at novelty detection, and a set of measured claims that the code previously asserted without evidence.

1 Structure of the model

1.1 Where the code lives

Four modules, in dependency order:

- `model/evidential_modules.py` — the Dempster–Shafer primitives: `Distance_layer`, `DistanceActivation_layer`, `Belief_layer`, `Omega_layer`, `Dempster_layer`, `DempsterNormalize_layer`, assembled into `Dempster_Shafer_module`; plus the two readouts (`DM`, `pignistic_probability`) and the two losses (`EvidentialLoss`, `PignisticNLLLoss`).
- `model/eucr_backbone.py` — `EucrResNet1D`, a ResNet-18 style 1D backbone whose linear `fc` head is replaced by per-task DS heads, with an `EvidentialProbe` hanging off each of the four residual stages.
- `model/eucr_consolidation.py` — importance estimation (`compute_importance`), channel pooling (`to_channel`), cross-task accumulation (`accumulate`), the weight anchor (`snapshot`) and the quadratic penalty.
- `model/eucr.py` — the learner `Net` that the La-MAML harness drives: `observe` for a training step, `finalize_task_after_training` for the end-of-task consolidation.

1.2 Data and backbone

Inputs are 1024-sample flat IQ vectors, reshaped to two channels (2, 512) (I and Q), min-max scaled per feature using training-split statistics. The backbone is the standard ResNet-18 arrangement in 1D: conv1/BatchNorm/ReLU/maxpool, then four stages of two BasicBlock1D each at widths 64, 128, 256, 512, global average pooling, and a LayerNorm on the pooled 512-d feature. Dropout ($p = 0.2$) sits between stages.

An important property of the data: roughly half of every task’s samples carry the label -1 , meaning *noise* rather than any emitter class (48% for **t2-rcn**, exactly 50% for the **deepgrad** files). The task-incremental loader remaps each task’s labels into a contiguous block and, in the default (non-collapsed) mode, appends noise as the *last* class of that block: `neg_label = labels_offset + unique_labels.size`. Evaluation macro recall then excludes that class. This matters for reading results: a model that predicts the noise class for every sample scores exactly 0.000, because every counted class has zero recall. The alarming zeros in the June logs are that degenerate state, not a numerical failure.

1.3 The Dempster–Shafer head

A DS module maps a feature vector $f \in \mathbb{R}^F$ to a mass function over the singleton classes plus the frame Θ (“ignorance”), i.e. a vector in $\mathbb{R}^{C+1}$. It does so through P prototypes, $P = C \cdot \text{proto_factor}$:

1. **Distance.** $d_p = 1 - \cos(f, w_p)$ for each prototype w_p (cosine is the default; the pooled features are LayerNorm’d, so their norm is near-constant and squared-Euclidean distance is dominated by that constant).
2. **Activation.** $s_p = \alpha_p \exp(-\gamma_p d_p)$ with $\gamma_p = \eta_p^2$ and $\alpha_p = \sigma(\xi_p)$, followed by a division by $\max_p s_p$.
3. **Belief.** Prototype p splits its activation over classes with weights $u_{cp} = \beta_{cp}^2 / \sum_{c'} \beta_{c'p}^2$, giving a mass function $m_p(c) = u_{cp} s_p$ and $\omega_p = 1 - s_p$.
4. **Fusion.** The P prototype mass functions are combined by Dempster’s rule and normalised.

For singleton-plus-ignorance masses, Dempster’s rule for two sources is

$$m(c) \propto m_1(c)m_2(c) + m_1(c)\omega_2 + \omega_1 m_2(c), \quad \omega \propto \omega_1 \omega_2. \quad (1)$$

Two readouts sit on top. DM produces per-class expected utilities $m(c) + (1 - \nu)\omega$ with $\nu\omega$ retained in the last column, trained by **EvidentialLoss** (per-class BCE plus a gated KL warm-up). **pignistic_probability** produces the Smets transform $\text{BetP}_c = m(c) + \omega/C$, a genuine distribution, trained by **PignisticNLLLoss**. The task-incremental configuration uses the pignistic head.

1.4 Per-task heads, shared probes

EucrResNet1D owns one DS head per task, each over only that task’s class count, and routes `forward(x, t)` through head t , scattering its scores into the global logit vector at the task’s offsets. A head is trained only during its own task and receives no gradient afterwards, so it is frozen by construction; the shared backbone is what consolidation has to protect.

The four stage *probes* are the opposite: they are shared across tasks and built at the *global* class width n_{outputs} , and their loss is not masked to the current task. Each probe average-pools its stage feature map, LayerNorms it, and runs its own DS module. Their combined loss enters the objective at weight `probe_loss_weight`.

1.5 Consolidation

After task t , `compute_importance` differentiates the mean backbone DS uncertainty — selectable as non-specificity (ω), discord ($H(\text{BetP})/\log C$), or their sum — with respect to every shared backbone parameter, accumulating $\|\nabla\|^2$ over the task’s data. The result is optionally pooled to one score per convolution output channel, normalised to unit mean, and summed across tasks. During task $t+1$ the loss gains $\lambda \sum_i \Omega_i (\theta_i - \theta_i^*)^2$. No pruning, no masks. Despite the shape of the formula this is *not* a Fisher approximation: there is no

Laplace / log-likelihood-curvature story behind it. Squaring the gradient of a scalar *output functional* over effectively unlabelled data is Memory Aware Synapses (Aljundi et al., ECCV 2018, $\nabla\|F(x)\|^2$), with a DS uncertainty substituted for MAS’s output norm. It should be described as a MAS variant and benchmarked against MAS.

1.6 The training step

`Net.observe` runs one forward pass with the probes, forms

$$\mathcal{L} = \mathcal{L}_{\text{head}} + w_{\text{probe}} \overline{\mathcal{L}_{\text{probe}}} + \lambda \mathcal{R}_{\text{consolidation}}, \quad (2)$$

and steps two parameter groups separately: the shared backbone at `lr`, and the evidential parameters (heads, probes, and the distance-activation η, ξ) at a fraction of it. Each group is gradient-clipped against its own budget so the high-gradient head cannot scale the backbone’s update to nothing. Mixed precision is disabled in three independent places — `amp: false` in the config, a hard override in `main.py`, and an `autocast(enabled=False)` guard inside the step — on the stated grounds that the DS head’s exponential and normalisation chains are unstable in reduced precision. Section 5.1 tests that claim.

1.7 Knobs added during this study

Five options were added while establishing the results below. All default to the previous behaviour except where stated.

- `--eucr_anchor_mode {loss, proximal}` — the anchor’s form (Section 2.2). Default `loss`, but `proximal` is the recommended setting and the shipped config now uses it.
- `--eucr_bn_stats {batch, running, freeze, per_task}` — the normalisation-statistics protocol (Section 2.1). **Default changed** to `batch`, which matches the harness.
- `--eucr_uncertainty {..., uniform, random_proj}` — two ablation controls on the importance readout (Section 3).
- `--eucr_belief_init {random, class}` — round-robin prototype-to-class initialisation of β .
- `--eucr_ce_aux_weight, --eucr_ds_detach` — an auxiliary per-task linear cross-entropy head sharing the backbone, and an option to stop the DS head’s gradient at the pooled feature.
- `--eucr_temper, --eucr_activation_norm, --eucr_readout_scale` — the cautious-combination family used to revive the ignorance mass in Section 4.3.

2 Two measurement faults and one numerical fault

2.1 EUCR was scored on a different protocol from its baselines

`ResNet1D.forward` carries a `bn_training` parameter defaulting to `True`, and its body is

```
prev = self.model.training
self.model.train(bn_training)
try:    ...
finally: self.model.train(prev)
```

so it forces its inner module into training mode on *every* call. Verified directly: after `model.eval()`, a forward still reports `bn1.training == True`. Every `ResNet1D`-based learner in the repository — EWC, SI, ER, GEM, LwF — therefore evaluates with *batch* normalisation statistics, transductively. No evaluation path passes `bn_training=False`; the 21 call sites that do are importance passes, replay encoding and distillation inside `woe_si`. This is a known and deliberate property: the `--eval-bn` help text in

`scripts/confusion.from.run.py` already records that “every reported number in this project uses batch statistics”.

`EucResNet1D` had no such wrapper, so EUCR inherited whatever `model.eval()` set — frozen running statistics carried over from the last task trained. It was the only model in the table on that protocol. Since consecutive tasks are different radar datasets, the two protocols differ a great deal. Re-scoring existing checkpoints, with no retraining (three seeds):

protocol	EUCR			EWC		
	diag	final	BWT	diag	final	BWT
<code>batch</code> (harness)	0.645	0.600	−0.060	0.713	0.428	−0.380
<code>running</code> (frozen)	0.616	0.293	−0.432	0.678	0.339	−0.452
per-task buffers	0.616	0.365	−0.335	0.678	0.198	−0.639
AdaBN recalibration	0.647	0.599	−0.063	0.681	0.434	−0.330

For EWC the protocol barely matters (0.428 against 0.434); for EUCR it is worth 0.31 final F1 and 0.37 BWT. `EucResNet1D.forward` and `Net.forward` now take `bn_training`, and `--eucr_bn_stats` defaults to `batch`.

Two caveats belong with this. Lifting task j ’s *stored* buffers is not a valid per-task protocol — they were estimated under task j ’s weights, so pairing them with later weights describes a different network, which is why that row is the worst for EWC. AdaBN recalibration (re-estimating statistics on task j ’s data with the current weights, forward passes only) is the correct version, but it requires retaining data from each past task, which is a replay assumption a regularisation-only method rejects. And `batch` is transductive: a sample’s prediction depends on the others in its evaluation batch. Swept over test batch sizes 32/64/128/512, the effect is real but mild (task-0 recovery 0.731/0.742/0.748/0.769 on one seed). There is no option here that is both non-transductive and storage-free.

2.2 The anchor’s loss form is numerically unstable

Explicit descent on $\lambda \Omega (\theta - \theta^*)^2$ has curvature $k = 2\lambda\Omega$ and is stable only while $\text{lr} \cdot k < 2$. `compute_importance` normalises Ω to unit *mean*, but the distribution is extremely heavy tailed, so the mean is set entirely by the tail:

	p50	p90	p99	p99.9	p99.99	max
Ω	2.7×10^{-4}	3.5×10^{-3}	0.42	23.7	1809	1.7×10^4

The median is $3600\times$ below the mean. At the shipped $\lambda = 0.09$ and $\text{lr} = 0.01$, 490 coordinates sit outside the stability window with $\text{lr} \cdot k = 30.7$ at the maximum. Measured on a batch perturbed 0.01 from θ^* :

λ	$\ g_{\text{task}}\ $	$\ g_{\text{anchor}}\ $	ratio	clip factor
0.09	5.13	311	61	0.0158
1	5.24	4.6×10^3	886	1.1×10^{-3}
10	5.00	5.8×10^4	1.2×10^4	8.6×10^{-5}
1000	4.59	6.5×10^6	1.4×10^6	7.7×10^{-7}

Because `clip_grad_norm` rescales by one global scalar, a handful of tail coordinates drag the whole network’s effective learning rate down: at the shipped λ the task gradient reaches the optimiser at 1.6% of its size. This explains a result that otherwise makes no sense — raising λ made *both* diagonal and final F1 worse, rather than trading one for the other.

The fix is the proximal (backward-Euler) form, which evaluates the anchor gradient at the new point and solves in closed form to a convex combination:

$$b = 2\text{lr} \lambda \Omega, \quad \theta_{\text{new}} = \frac{\theta^+ + b\theta^*}{1 + b}. \quad (3)$$

The mixing weight $b/(1 + b)$ saturates at 1 for any Ω , so the update can never overshoot. Applied after the optimiser step, with the penalty removed from the backward pass entirely.

	diagonal F1	final F1	BWT
no anchor	0.7173 ± 0.0068	0.3261 ± 0.0193	-0.391
loss form, $\lambda = 0.09$	0.6265 ± 0.0145	0.5231 ± 0.0533	-0.103
proximal, $\lambda = 500$	0.6317 ± 0.0058	0.5695 ± 0.0179	-0.062
proximal, $\lambda = 5000$	0.6019 ± 0.0042	0.5932 ± 0.0038	-0.009
proximal, $\lambda = 50000$	0.5674 ± 0.0032	0.5679 ± 0.0028	+0.001

Three things change at once. Final F1 rises +0.070; the λ sweep becomes *monotone*, tracing a proper stability-plasticity frontier instead of degrading at both ends; and the seed spread falls $14\times$, from ± 0.0533 to ± 0.0038 . That last point retires a claim made repeatedly in earlier drafts of this note — that EUCR’s large seed variance was a coherent property of prototype-based evidential heads. It was substantially this bug. Note that proximal’s useful λ is far larger than the loss form’s, because b carries a factor of lr : unit Ω gives $b = 1$ at $\lambda = 1/(2\text{lr}) = 50$.

3 What the consolidation signal actually is

With a stable anchor the question “does the evidential weighting contribute anything” becomes answerable for the first time. It could not be asked under the loss form, because the clip factor depends on $\|g_{\text{anchor}}\|$ and hence on Ω ’s tail — a uniform Ω has no tail, so the two arms were being attenuated by different amounts and the comparison was confounded.

Four importance sources were swept over λ and compared as *frontiers*, because matched λ is not matched pull when the Ω distributions differ:

- **evidential** — the shipped readout.
- **uniform** — $\Omega_i = 1$ everywhere, i.e. plain L2-SP.
- **frozen probes** — probes never trained (`probe_loss_weight 0` gives them no gradient at all; verified, drift = 0.0000 across the sequence), still read through the DS fusion.
- **random projection** — MAS with a random head: the gradient of $\|Wf_s\|^2$ for a fixed random Gaussian W on the same pooled, normalised stage features. Everything the probes do is kept except the DS fusion and its learned parameters.

importance source	peak final F1	at diagonal
random projection	0.5962 ± 0.0015	0.604
evidential	0.5932 ± 0.0038	0.602
frozen probes	0.5901 ± 0.0065	0.596
uniform Ω	0.5791 ± 0.0011	0.577

The evidential and random-projection frontiers are not merely close at their peaks: they are *the same curve, shifted one decade in λ* .

position on frontier	evidential (diag / final)	random proj. (diag / final)
high plasticity	$\lambda=5$: 0.697/0.390	$\lambda=0.5$: 0.703/0.386
mid	$\lambda=500$: 0.632/0.570	$\lambda=50$: 0.641/0.569
optimum	$\lambda=5000$: 0.602/0.593	$\lambda=500$: 0.604/0.596
saturated	$\lambda=50000$: 0.567/0.568	$\lambda=5000$: 0.566/0.567

Every paired row matches to within ± 0.01 on both axes. The decade offset is predicted by the Ω statistics: the random readout’s median Ω is $57\times$ the evidential median, so the same pull $b = 2\text{lr}\lambda\Omega$ arrives at roughly $1/57$ the λ . And this is not a relabelling: the two Ω vectors have Spearman correlation 0.546, so about half the coordinate ranking differs — yet the frontier is identical.

The conclusion has three parts, and only the first is positive.

1. **The anchor does the work.** +0.15 final F1 over no anchor at all.
2. **Per-coordinate weighting adds a little.** +0.017 over uniform Ω (0.5962 against 0.5791, $\sigma \approx 0.001$ –0.002; separable).
3. **None of that weighting need be learned, and none of it need be evidential.** Evidential, frozen-DS and random-projection are mutually indistinguishable (within ± 0.003 against $\sigma \approx 0.002$ –0.007). Ω is a layer-and-channel sensitivity measure that any random readout recovers.

This agrees with the per-coordinate result in Section 4.4, where a displacement control beat every importance measure on a restoration test.

4 The evidential content

4.1 The fusion admits a closed form

`Dempster_layer` applied Dempster’s rule in a Python loop, fusing prototypes one at a time: $P-1$ sequential iterations with P as large as 138. Adding the ignorance column to a class column gives the commonality $q_k(c) = m_k(c) + \omega_k$, and the rule is then exactly $q(c) = q_1(c)q_2(c)$ alongside $\omega = \omega_1\omega_2$. Both are products, so fusing P sources is two reductions:

$$\log q(c) = \sum_p \log(m_p(c) + \omega_p), \quad \log \omega = \sum_p \log \omega_p, \quad m(c) = q(c) - \omega. \quad (4)$$

The loop’s per-step renormalisation is a positive per-sample scalar the recursion carries through linearly, so it only rescales the result — this is not an approximation. Outputs agree with the loop to 2×10^{-6} and gradients to 5×10^{-13} ; the layer is $9.6\times$ faster at $P = 18$, $46\times$ at $P = 81$ and $78\times$ at $P = 138$, and an epoch falls from 8.9s to 1.35s, matching EWC’s per-epoch cost. Working in logs with a max-shift also removes the underflow that P -fold products invite.

4.2 The ignorance mass is zero by construction

The measured ω is $\sim 10^{-16}$ at the head and 10^{-14} to 10^{-25} at the probes. Three independent constructions cause this, not one. (i) `DistanceActivation_layer` ends with $s \leftarrow s/(\max_p s_p + 10^{-4})$, pinning the best prototype at $s \approx 1$ and its ignorance at the 4.2×10^{-4} rounding artefact. (ii) The P -fold product $\omega = \prod_p (1 - s_p)$. (iii) Cosine distance is bounded in $[0, 2]$, so with $\gamma \approx 1$ and $\alpha = \sigma(0) = 0.5$ activations live in $[0.068, 0.5]$ and no input, however novel, can drive $s_p \rightarrow 0$. Denoeux’s ENN relies on unbounded Euclidean distance precisely so novel inputs send mass to Θ .

Measurement makes (iii) worse than the bound suggests. After training, $\gamma \in [0.95, 1.08]$ and $\alpha \in [0.498, 0.502]$ — essentially unmoved from initialisation — and this is *not* gradient starvation: gradient norms on η and ξ (1.9 – 5.6×10^{-2} , 1.1 – 2.0×10^{-2}) are the same order as on the prototypes and belief weights (4.8 – 7.2×10^{-2} , 3.6 – 7.5×10^{-2}). The layer gets a normal gradient and does not move. Observed cosine distances span only $[0.60, 1.34]$, so no prototype ever approaches a feature and $\prod_p (1 - s_p)$ is nearly constant in the input.

Two consequences follow, both verified. `eucr_uncertainty=both` is *bit-identical* to `discord` ($\max |\text{both} - \text{discord}| = 0$), so that flag has two live settings, not three. And with $\omega \approx 0$, $\text{BetP}_c = q(c) - \omega(C-1)/C$ reduces to $q(c)$, so by Eq. (4) the normalised output is *exactly*

$$\text{BetP} = \text{softmax}\left(\sum_p \log(1 - s_p(1 - u_{cp}))\right), \quad (5)$$

measured to $\max |\text{BetP} - \text{softmax}| = 1.8 \times 10^{-7}$ with argmax agreement 1.0000. In its shipped configuration the DS head *is* a softmax classifier over an unusual, saturating, non-negatively-weighted log-linear score.

An important distinction, since an earlier draft of this note got it wrong: $\nabla \omega \propto \omega$, so although ω ’s *value* is $\sim 10^{-14}$, its relative sensitivity *pattern* is well defined and survives the unit-mean normalisation. *nonspecificity* is dead as a *value* (novelty detection) but alive as an *importance pattern*, and performs the same as `discord` in that role.

4.3 A pre-registered gate: does the formalism earn its place?

Given Eq. (5), EUCR has a claim to be more than a softmax only if the ω and K columns can be revived *and* then carry information a softmax does not. Pre-registered as a gate; both halves came back negative.

Revival works. Three changes together: drop the max-division (`--eucr.activation.norm none`); temper the fusion by dividing both log-sums by P^τ (`--eucr.temper`), the geometric mean of commonalities at $\tau = 1$, which stops P prototypes reading *one* feature vector being counted as P independent sources; and rescale the decision logits by P^τ (`--eucr.readout.scale untemper`). Median ω rises from 10^{-20} to 0.78. The third change is essential rather than cosmetic: tempering shrinks the readout logits by P^τ , and without undoing that on the decision, $\tau = 1$ trains to macro recall 0.000 against 0.710 with it. Tempering appeared untrainable; it was only cold. $q_p(c) \geq \omega_p$ survives geometric means, so $m(c) \geq 0$ holds for any τ .

A confound had to be removed first. Every task is $\sim 50\%$ noise, and although the in-distribution set excluded noise *rows*, the head still had a noise *output* that absorbs anything unfamiliar. Noise-prediction rates on cross-file OOD were 0.04/0.64/0.76 across seeds against 0.02/0.11/0.18 in-distribution. Retraining with noise dropped entirely roughly doubles every score on the held-out-class test and *annihilates* the cross-file column (all scores 0.46–0.56, the noise floor). Cross-file novelty was never being detected; the noise class was.

It buys nothing. Held-out-class novelty (train on three of `t2-rcn`’s signal classes, score the other two), noise-free, signed AUROC, three seeds at matched accuracy:

score	held-out-class	$ \cdot - 0.5 $
$\log \omega$ (DS)	0.274 ± 0.035	0.226
$\log K$ (DS)	0.721 ± 0.044	0.221
max BetP (softmax)	0.230 ± 0.027	0.270
entropy (softmax)	0.227 ± 0.025	0.273

The softmax baselines are the strongest scores on the table. Under a true null the $|\text{AUROC} - 0.5|$ statistic has expectation $\approx 0.8\sigma$, so for the observed per-seed σ the floor is 0.02–0.12. Signed AUROC is reported because almost every score is *anti*-correlated with novelty: unseen emitters are classified *more* confidently and with *less* ignorance than seen ones. $\log K$ is the only correctly-signed score — its magnitude ties rather than beats, but a score thresholdable in the natural direction is operationally worth more than an equally strong one that must be inverted. That is the one thread of the formalism that survives, and it is thin.

And it costs accuracy. On the four-task benchmark, tempering with ω revived gives final F1 0.329 (nonspecificity) and 0.386 (discord) against 0.51–0.53 with ω dead. The configuration in which the evidential quantity is meaningful is the worst-performing one.

4.4 PR-E1: a displacement control beats every importance measure

Pre-registered. On task-0 held-out data at end of sequence, restore the top- $k\%$ of shared backbone coordinates by a measure’s rank to their task-0 values and measure the fraction of the loss recovered, relative to a statistics-restored baseline. At the primary $k = 1\%$, three seeds:

measure	$R(1\%)$
$ \Delta\theta $ (control)	$+0.699 \pm 0.104$
Fisher $\times \Delta\theta^2$	$+0.682 \pm 0.161$
MAS-entropy $\times \Delta\theta^2$	$+0.579 \pm 0.020$
Fisher	$+0.515 \pm 0.073$
MAS-entropy (shipped)	$+0.334 \pm 0.222$
$ \Delta\theta \cdot g $	$+0.318 \pm 0.248$
$ \theta_0^* $ (control)	$+0.103 \pm 0.152$
random (control)	$+0.006 \pm 0.007$

The gate passes ($R_{\text{MAS}} - R_{\text{random}} \geq 0.10$ in 3/3 seeds, sanity kill not triggered), but it was the wrong comparison: forgetting is localisable — 1% of coordinates restores 70% of it — yet the coordinates are identified by *how far they moved*, not by any importance functional. Multiplying either importance by $\Delta\theta^2$ moves it most of the way to $|\Delta\theta|$ and neither product beats it, so the $\Delta\theta^2$ factor carries the signal. One caveat that cuts against the headline: $R(k)$ rewards L2 proximity to θ_0^* , which $|\Delta\theta|$ maximises by construction, so the products rather than the control are the fair test — and they say the importance term buys nothing.

5 Other measured properties

5.1 Mixed precision: lossy, not unstable

The comment justifying the `autocast(enabled=False)` guard asserted the head is “unstable in AMP”. Neutralising the guard and training for real: **zero** non-finite losses and zero skipped steps across nine four-task runs and three single-task runs, in both bf16 and fp16. Paired per-seed diagonal F1 against fp32: bf16 -0.032 ± 0.031 (not significant), fp16 -0.085 ± 0.012 (significant, but these runs use no `GradScaler`, so an unknown share is missing loss scaling). AMP buys $\sim 5\%$ epoch time ($6.19\text{s} \rightarrow 5.89\text{s}$), because after the Dempster fix the model is dominated by small kernel launches rather than matmul FLOPs. **Keep the guard, for the cost/benefit reason, not the stability one.** Note that layer-level precision diagnostics rank the dtypes the *opposite* way from the end-to-end result (bf16 gradient cosine to fp32 is 0.32–0.50 against fp16’s 0.976–0.998), so they did not predict end-to-end damage here.

5.2 Two architectural choices that are load-bearing

- **Euclidean distance destroys the model:** diagonal and final F1 both 0.0974 ± 0.0001 with BWT exactly 0.000 on all three seeds — it never learns. The pooled features are `LayerNorm`’d, so their L2 norm is near-constant and squared-Euclidean distance is dominated by that constant. Cosine is not a tuning choice.
- **GroupNorm destroys the model:** 0.0981 ± 0.0001 , training accuracy 0.00, loss pinned at 2.23. BatchNorm’s batch statistics are load-bearing for the cosine-prototype head.

5.3 Ablations at the corrected protocol

Three seeds, loss-form anchor at $\lambda = 0.09$, so these sit below the proximal numbers but are internally comparable:

	diagonal F1	final F1	BWT
default	0.6265 ± 0.0145	0.5231 ± 0.0533	-0.103
<code>probe_loss_weight 0</code>	0.6245 ± 0.0152	0.5515 ± 0.0150	-0.073
<code>eucr_head dm</code>	0.6292 ± 0.0082	0.5031 ± 0.0519	-0.126
CE aux head, $w = 0.3$	0.6313 ± 0.0113	0.5471 ± 0.0216	-0.084
CE aux head, $w = 1$	0.6486 ± 0.0093	0.5248 ± 0.0400	-0.124
CE aux head, $w = 1$, DS detached	0.7440 ± 0.0078	0.4132 ± 0.0254	-0.331

Two entries deserve comment. Removing the probe loss *improves* final F1 — and it does more than stop training the probes, it freezes them at random initialisation entirely (verified: drift 0.0000 across the sequence, against 0.6225 when trained), because nothing else feeds them. The training loss traces are indistinguishable, so the probe loss was not contributing to representation learning either. And detaching the DS head so cross-entropy alone shapes the backbone lifts the diagonal to 0.744, *above* EWC’s 0.711: the DS head does reach control-level plasticity when it does not have to shape the representation itself. It pays for that in retention.

6 Results

Four tasks (t2-rcn, t0-rcn, t0-deeprad, t1-deeprad), 10 epochs, 10 000 samples per task, three seeds (0, 39, 55), all on the batch protocol.

	diagonal F1	final F1	BWT
LwF	0.7069 ± 0.0055	0.6602 ± 0.0170	−0.047
EWC	0.7113 ± 0.0034	0.4270 ± 0.0251	−0.284
EUCR, no anchor	0.7173 ± 0.0068	0.3261 ± 0.0193	−0.391
EUCR, loss anchor $\lambda = 0.09$	0.6265 ± 0.0145	0.5231 ± 0.0533	−0.103
EUCR, proximal $\lambda = 5000$	0.6019 ± 0.0042	0.5932 ± 0.0038	−0.009
EUCR, proximal + random Ω	0.6039 ± 0.0070	0.5962 ± 0.0015	−0.008

EUCR sits clearly above EWC on final F1 (+0.17) and clearly below LwF (−0.06), and is behind both on diagonal. It has the flattest BWT of any method here, which is the axis its consolidation targets.

7 Positioning the method

7.1 What EUCR is, stated honestly

An evidential prototype head with per-task DS heads and a MAS-style importance anchor. On this benchmark it is a working continual learner that beats EWC comfortably and loses to LwF. Its retention is excellent and its plasticity is its weakness — diagonal 0.60 against 0.71 for both baselines.

The parts that demonstrably contribute are: the quadratic anchor (+0.15), applied in proximal form (+0.07 on top, plus a 14× variance reduction), with *some* per-coordinate weighting (+0.017), and per-task heads that are frozen by construction. The parts that do not contribute, on measurement, are the Dempster–Shafer fusion as a source of that weighting, the deep evidential supervision on the probes, and both uncertainty readouts as novelty signals.

That is a narrower method than the original framing, but it is a defensible one, and the accounting is more useful to a reader than the claim would have been.

7.2 Report the evaluation protocol

The single most consequential number in this note was produced by neither the model nor the data but by which normalisation statistics the evaluation used. Any paper using this harness should state that its numbers are computed with batch statistics, and therefore transductively, and should report at least one alternative. The three options are set out in Section 2.1; none is both non-transductive and storage-free.

7.3 Calibrate λ against the stability window, not by search

Ω ’s heavy tail means the effective anchor strength is set by a few coordinates. Under the loss form, check $\max_i \text{lr} \cdot 2\lambda\Omega_i < 2$ before trusting any λ ; under the proximal form this is unnecessary by construction, which is the main argument for preferring it. Reporting the ratio $\|g_{\text{anchor}}\|/\|g_{\text{task}}\|$ alongside λ would have made this visible immediately.

7.4 Compare importance schemes as frontiers

Matched λ is not matched pull when the Ω distributions differ — uniform Ω 's median is $3600\times$ the evidential median. Every point-comparison of importance schemes in earlier drafts of this note was therefore uninterpretable. Sweep λ and compare the (diagonal, final) trade-off curves.

7.5 Where the remaining plasticity is

The detached CE arm reaches diagonal 0.744, so the representation the DS head needs is reachable; the head simply does not shape it well on its own. That, plus the γ/α layer never training despite receiving normal gradients, localises the plasticity deficit in the distance-to-activation parameterisation rather than in the evidential formalism. It is the most promising remaining direction and the one this note does not settle.

8 Reproducing

```
main.py --config configs/base.yaml --config configs/models/til/eucr.yaml \
  --task-order-files "t2-rcn,t0-rcn,t0-deeprad,t1-deeprad" \
  --samples_per_task 10000 --n_epochs 10 --expt_name <arm>
```

About two minutes per arm; several fit on one A6000 concurrently. Every invocation is a three-seed sweep — read all three per-seed `results.txt`, never seed 0 alone. Decision rules and amendments for the pre-registered gates are in `docs/eucr/preregistration.md`.